Operations Manual

Discount Optimization System — End to End

Architecture · Execution runbooks · Weekly operations
Learning loop · Decision rules · Troubleshooting

1 · Stop over-investment

CI break-even + DML → cut list, executed in guarded 3-point steps

2 · Find under-investment

reliably-pays cells → reinvest pilots with control cities

3 · Allocate the budget

portfolio optimizer under budget, glide, ladder and floor constraints

4 · Learn every week

frozen baselines → actuals → scorecard → kill-switch → retrain

One sentence: raw exports become a clean fact table; two independent engines recommend; the tracker executes only what both agree on, in small guarded steps; every week the register grades the model — and anything that hurts is automatically reverted.

Version: 7 July 2026 · written against git HEAD (6fd3f1e) · 24 Mantra Organic × Blinkit · 84 SKUs × 11 cities = 585 cells

1 · What this system is, and why each part exists

You run one brand (24 Mantra Organic) on one platform (Blinkit): 84 SKUs × 11 cities = 585 cells (a cell = one SKU in one city — the atomic unit of every decision). The system answers four business questions; every module maps to one of them:

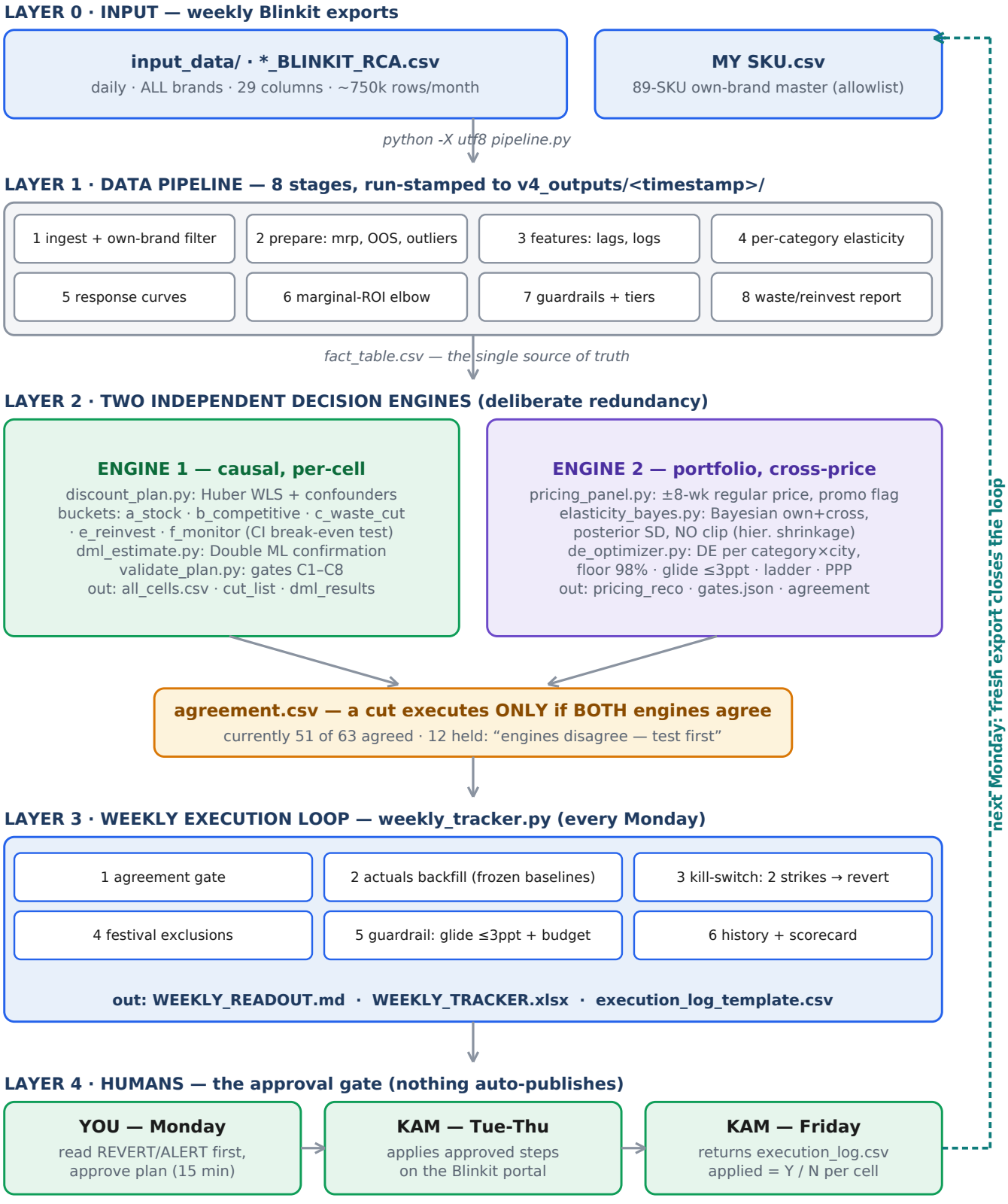
#	Business question	Subsystem that answers it
1	Where am I over-investing in discount (paying for volume that would come anyway)?	Engine 1: confounder-controlled regression → buckets → DML confirmation → cut list
2	Where would MORE discount pay?	Reinvest detection (CI test + DML headroom) → pilot protocol
3	How should a fixed discount budget be spread across cells?	Engine 2: DE optimizer (cross-price aware) + guardrail budget cap (full allocator: planned)
4	Is the model right, week after week — and does it improve?	Weekly tracker: frozen baselines → actuals → scorecard → kill-switch → 4-weekly retrain

Design principles (inherited from PepsiCo's PricingAI):

ML demand estimation feeding a constrained optimizer · validation gates before any model is trusted · human approval before any execution · thresholds externalized in config, not buried in code · honest uncertainty — low-confidence cells are tested, never banked.

2 · The architecture — end to end

Data flows top to bottom; the dashed teal line closes the weekly loop. Engine 1 asks “is this cell's discount causally wasted?”; Engine 2 asks “what happens to the whole portfolio?” — a cut executes only when both say yes (deliberate redundancy, like two cockpit instruments).



3 · Data contracts — the files that matter

Every file below is a handoff between two parts of the system (or between the system and a human). If a file is missing or malformed, the consumer column tells you what breaks.

File	Grain / key content	Produced by	Consumed by / why
input_data/*_RCA.csv	product×city×day · 29 cols · all brands	You (portal export)	stage1 — raw truth
MY SKU.csv	89 rows: own-brand allowlist	You (static)	stage1 — brand filter
fact_table.csv	cell×day + stable_mrp, selling_price, discount, is_regular_day, OOS/festival flags	stage2	everything — the cleaned single source of truth
plan/all_cells.csv	one row per cell: bucket, confidence, cur/tgt discount, net_gain, reason	discount_plan.py	tracker + pricing engine — Engine 1's verdict
plan/dml_results.json	per category: θ, SE, waste?, ₹ save	dml_estimate.py	validate C8 + reinvest — causal certificate
pricing/agreement.csv	585 rows: pricing_action, agree_with_cut	pricing_engine.py	tracker — the two-engine handshake
pricing/gates.json	R^2, wMAPE, bias, band checks per category	elasticity module	you — model quality certificate
tracker_history.csv	cell×week: predictions, actuals, applied, strikes, cell_status, baselines	tracker	scorecard + kill-switch — the register (system memory)
baselines.json	cell → frozen pre-action baseline (4-wk mean)	actuals.py (once)	backfill — the fixed yardstick; never re-computed
execution_log_template.csv → execution_log.csv	week, cell, action, applied Y/N	tracker emits; KAM fills	tracker — separates “model wrong” from “never applied”
WEEKLY_READOUT.md / .xlsx	the weekly decision document	tracker	you + KAM
competitor_features.csv	category×city×week comp price/discount (6,534 rows)	competitor_features.py	pending: DML challenger pass (R5)

Why baselines are frozen

If the “before” number re-computed every week, a cell that naturally drifts would fake wins or losses. So the pre-action baseline (last-4-week mean) is captured once, persisted in baselines.json, and every later week is measured against that fixed yardstick. Verified in code: backfill never overwrites a filled actual and never re-stamps a scored row's baseline.

4 · Runbook R0 — one-time sanity check

Run once before go-live and after any machine/sync change. Catches environment breakage in minutes (like the truncated-file sync damage found on 6 July).

```
git status && git diff --stat
git restore scripts/ stage*/ # only if code files show big deletions
python -X utf8 scripts/tracker/killswitch.py # → "All smoke-test assertions passed"
python -X utf8 scripts/tracker/actuals.py # → same
python -X utf8 scripts/pricing/de_optimizer.py # → same (needs scipy)
python -X utf8 scripts/tracker/verify_loop.py # END-TO-END → "LOOP CLOSED: YES"
```

⚠ verify_loop.py resets tracker_history / baselines / execution_log — run only before go-live or on a copy, never mid-season.

Runbook R1 — the Monday weekly run (~30–40 min machine, 15 min you)

STEP 1 · Export & drop data — you, ~10 min
Last week's RCA from Blinkit portal → input_data/ (never delete old months — elasticity needs history)

↓

STEP 2 · Refresh the fact table — 10–20 min
python -X utf8 pipeline.py → new v4_outputs/<ts>/fact_table.csv. Models are NOT refit weekly (that is R2).

↓

STEP 3 · Run the tracker with actuals — 2–5 min
python -X utf8 scripts/tracker/weekly_tracker.py --actuals v4_outputs/<NEW>/fact_table.csv
backfills actuals → kill-switch (applied cells only) → agreement gate → glide + budget → readout

↓

STEP 4 · Read the readout — you, 10 min
Order: ⚠ REVERT/ALERT block → budget status → this week's cuts → track record. Reverts outrank everything.

↓

STEP 5 · Approve & hand off — you, 5 min
Send KAM: WEEKLY_TRACKER.xlsx + execution_log_template.csv. Rule: only listed cells, exactly the listed step.

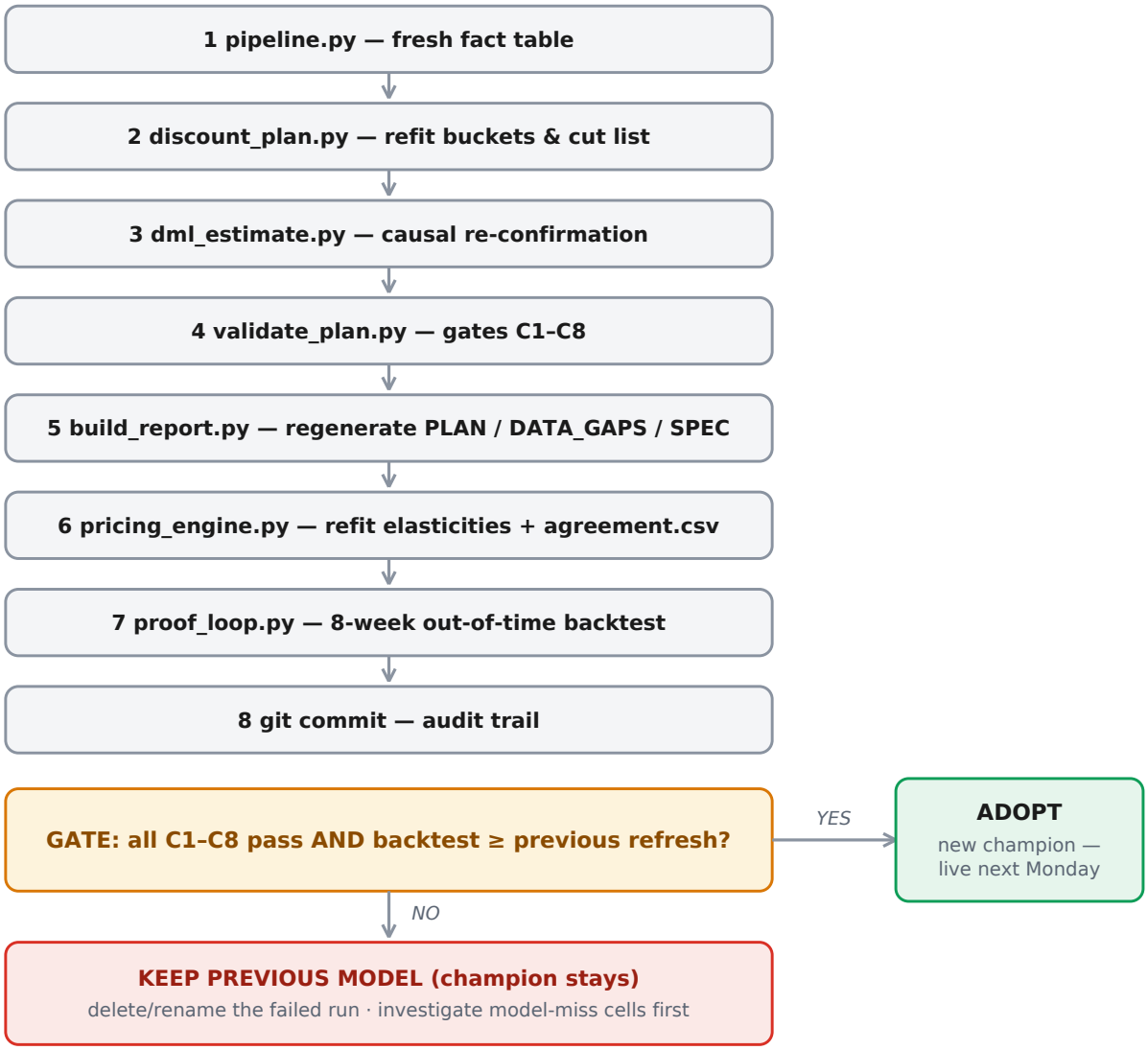
↓

STEP 6 · KAM applies & confirms — by Friday
Fills applied Y/N (+date, reason) → save as DISCOUNT_PLAN/execution_log.csv. No file = nothing gets scored.

Why weekly, not monthly: actuals must land within days of an action or the kill-switch reacts a month late. Why models are NOT refit weekly: weekly refits chase noise — execution runs weekly, learning runs 4-weekly (same cadence split PepsiCo uses). Optional flags: **--budget_pct 0.125** (set a REAL cap — the default merely equals current spend), **--week/--date** only to override auto-derivation.

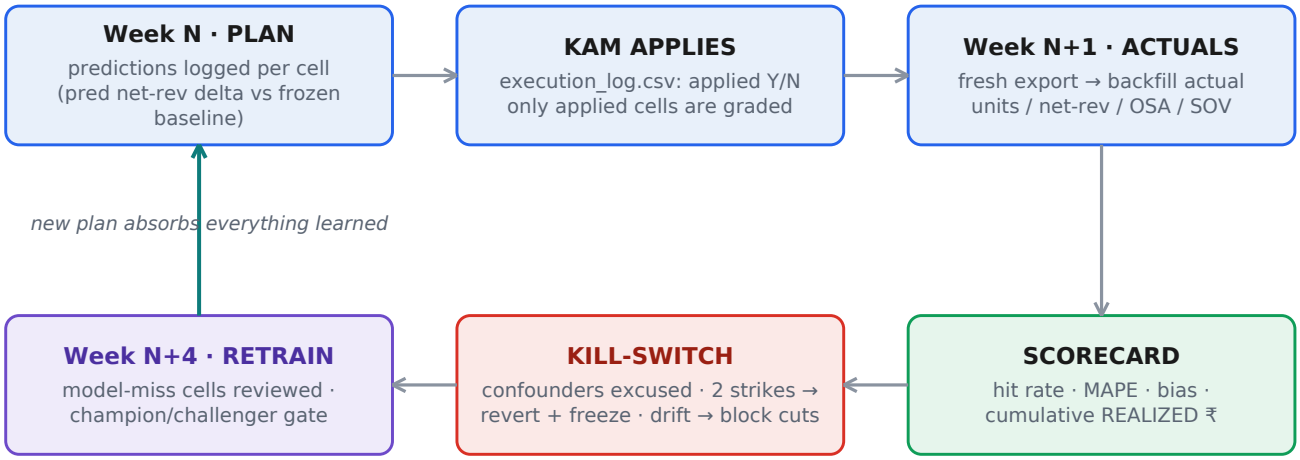
5 · Runbook R2 — the 4-weekly model refresh (the learning half)

Run after four new weeks of data, in exactly this order — each step feeds the next. The gate at the bottom is the champion/challenger discipline: a new model must EARN its job.



Acceptance rules (currently your manual job): (a) validate_plan passes all gates; (b) proof_loop's out-of-time accuracy is no worse than the previous refresh; (c) elasticity gates hold (own-price in (−2.5, 0), wMAPE < 0.40, |bias| < 10%). Also review the kill-switch's model-miss cells — they are the literal list of “where assumptions failed”.

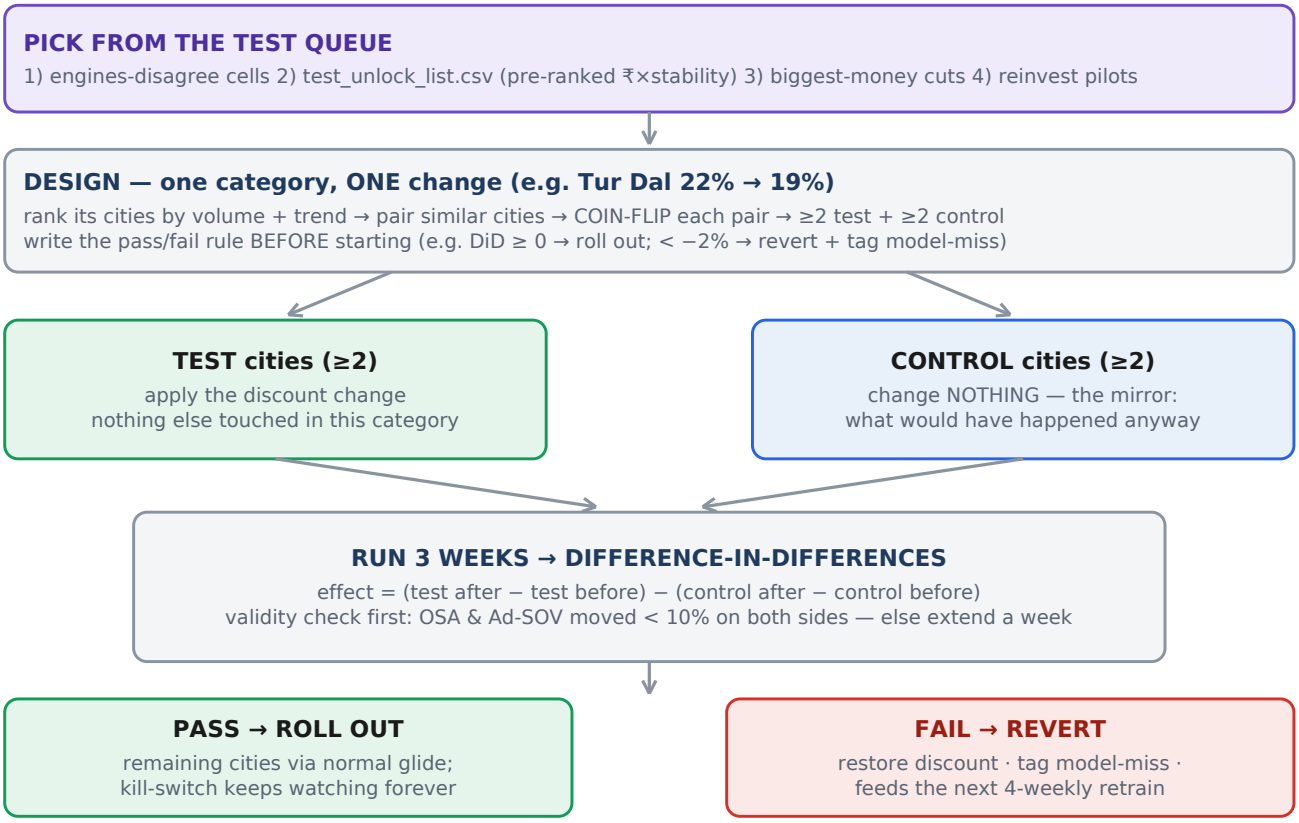
6 · The learning loop — how the system improves every week



Your objective-4 requirement	Mechanism	Status
Compare predicted vs actual sales	actuals backfill → scorecard MAPE / R^2 per week	built
Compare predicted vs actual uplift	pred vs actual net-revenue delta vs frozen baseline	built
Identify where assumptions failed	strikes + model-miss tags; confounded weeks excused (OSA/SOV −10%)	built
Learn from new data	R2 refresh; challenger replaces champion only if backtest improves	manual gate
Update recommendations automatically	next Monday's tracker consumes the newest valid plan automatically	built

7 · Runbook R4 — the geo-split test protocol

Used when the readout or the agreement gate says “test first”, and for any big-money claim before full scale. Capacity discipline: 3–4 concurrent tests max, one per category, ≤10% of weekly discount spend exposed. Register every test in DISCOUNT_PLAN/TEST_REGISTER.csv (test_id, hypothesis, cells, dates, rule, status — create on first use).



Runbook R3 — quarterly (30 min): refresh festival calendar (seasonality.py) · revisit --budget_pct · populate STRATEGIC_SKUS hero list (never auto-cut) · review DATA_GAPS.md · archive old v4_outputs (keep last 6 + any run referenced by history).

Runbook R5 — competitor pass (its own session, planned): fit Model B = Engine 1 + competitor controls beside untouched Model A; adopt B only if out-of-sample $R^2 \geq 0.75$ holds, C1–C8 + DML re-confirm pass, and competitor coefficients have sane signs. Deliverable: a delta report — which cells change bucket, how much of the ₹6.98L survives. The weekly loop does not wait for it.

8 · Every decision rule and threshold in one table

Rule	Value	Where it lives	Why this value
Weekly glide step	≤ 3 ppt	guardrail R1 + DE bounds	small enough to revert cheaply; reaches targets in ~12 weeks
Revenue-protective revert	scaled pred Δ < 0 → hold	guardrail R2	never take a step the model itself predicts loses money
Budget cap	--budget_pct (SET IT)	guardrail R3	blocks reinvest increases when over cap; never invents cuts
Budget status bands	GREEN ≤95% · AMBER 95–105% · RED >105%	guardrail	early warning before the cap actually binds
Kill-switch strike	units < pred×0.95 AND net-rev Δ < 0	killswitch.py	both legs required — a cheap miss that still makes money is not a failure
Confounder excuse	OSA or Ad-SOV < baseline×0.90	killswitch (first check)	an out-of-stock week says nothing about the discount
Revert + freeze	2 consecutive strikes → restore, freeze 4 wks	killswitch	one bad week is noise; two is a pattern; freeze stops thrashing
Drift brake	hit-rate < 60% over ≥30 scored cells → block NEW cuts	killswitch	if aim is off portfolio-wide, stop rollout and force a retrain
Cut eligibility (quadruple lock)	c_waste_cut AND reliably_waste (CI) AND Engine-2 agrees AND not STRATEGIC_SKU	discount_plan + agreement + tracker	four independent locks before any cut executes
Waste test	optimistic CI edge still below break-even 1/(100−d)	discount_plan.py	only bank what survives the favourable reading
Reinvest test	pessimistic CI edge clears break-even + headroom > 1 ppt	discount_plan.py	symmetric honesty for spending more
Elasticity gates	R^2 floor · wMAPE < 0.40 · |bias| < 10% · own ∈ (−2.5, 0)	gates.json	PepsiCo's published acceptance bands
DML confirmation	θ + 1.96·SE < break-even per cut category	dml_estimate → C8	cuts must survive nonlinear-confounder stripping
DE constraints	revenue ≥ 98% baseline · ₹/kg ladder · PPP · box 0–45%	de_optimizer.py	portfolio safety fences
Festival weeks	excluded from waste scoring; budget +50% allowance	seasonality.py	planned festive discounts are strategy, not waste
Geo-test decision	pre-registered DiD rule · 3 weeks · ≥2+2 cities	R4 protocol	evidence, not vibes

System-health KPIs (watch monthly)

Hit rate (target: trend toward ~85%, PepsiCo's acceptance-quality bar) · decision-model MAPE at 3-ppt bins · revenue bias (watch systematic over-promise) · cumulative realized ₹ vs the ₹6.98L/month claim · % recommendations applied (ops health) · reverts per month (model health).

9 · Failure modes & troubleshooting

Symptom	Cause	Fix
SyntaxError importing killswitch / tracker	sync-truncated files (seen 6 Jul)	git status → git restore ; re-run R0 smoke tests
“No all_cells.csv — run discount_plan first”	tracker launched before the analysis chain	run R2 steps 1–2
Readout says “scored cells: 0” forever	execution_log.csv missing or still named _template	get the KAM file in place; applied values must be Y/N
A week silently skipped	week label already in history (idempotent append)	intended — safe re-runs; to redo a week, delete its rows from tracker_history first
Everything marked confounded	OSA collapse (platform stock issue)	correct behaviour — fix availability before judging discounts
Drift brake ON immediately	scoring unapplied/hold cells (pre-fix behaviour)	fixed at HEAD (82ea05a); verify with the killswitch smoke test
Gates fail after a refresh	thin or odd new data; category churn	champion stays; inspect per-category coverage in gates.json
DE groups_failed > 0	a degenerate category×city group	usually 1-SKU groups — safe to ignore if small
Two engines disagree a lot	expected on confounded categories	by design: those cells route to R4 tests, not to cuts
Festival week looks terrible	calendar doesn't cover the date	update seasonality.py windows (R3); confounded weeks don't strike anyway

10 · Glossary

Cell — SKU × city — the decision unit (585 today).

is_regular_day — non-festival, non-OOS, non-outlier day — the only rows models train on.

Break-even β — 1/(100−d): the demand slope where one more discount point exactly pays for itself.

Posterior SD — the Bayesian uncertainty band around an elasticity; wide = act only via test.

PPP — psychological price points (₹49/99/199…) — small demand steps at thresholds.

Glide — move in ≤3-ppt weekly steps so every move stays cheaply reversible.

Strike / freeze — kill-switch bookkeeping: 2 strikes = revert, then a 4-week cooling-off.

DiD — difference-in-differences: (test change) − (control change) = causal effect.

Agreement gate — both engines must independently endorse a cut before execution.

stable_mrp — 90th-percentile MRP per SKU-grammage; a de-noised list price.

Bucket — Engine 1's diagnosis: a_stock (fix availability) · b_competitive (defend) · c_waste_cut (reliably wasted) · e_reinvest (reliably pays) · f_monitor (uncertain).

DML (Double ML) — cross-fitted gradient-boosting residualization; strips nonlinear confounding before estimating the discount effect (orthogonal θ, cluster-robust SE).

DE — differential evolution — population metaheuristic for non-convex objectives (the PPP steps make the surface discontinuous; gradient methods stall).

Ladder — bigger pack must be cheaper per kg — enforced and auto-repaired.

Frozen baseline — the fixed pre-action reference all actuals are compared against.

Drift brake — portfolio-level stop on new cuts when live hit-rate collapses.

Champion / challenger — a new model replaces the old only by beating it out-of-sample.

11 · What is still manual or planned (so this manual stays honest)

Item	What it is	Status
Budget allocator	“spend exactly X% of baseline revenue, allocated by marginal ROI” as a DE constraint + greedy-waterline Excel cross-check	planned build
Marginal-ROI ladder report	per cell: each discount step → units, net revenue, spend, marginal ROI, elbow flag (the brand-facing proof artifact)	planned build
Champion/challenger automation	R2's acceptance gate is currently you comparing proof_loop numbers by hand	manual
Test register	TEST_REGISTER.csv maintained by hand until test volume justifies tooling	manual
Competitor pass (R5)	champion/challenger with competitor controls; delta report on the ₹6.98L	next session
Your inputs	real --budget_pct · STRATEGIC_SKUS hero list · weekly export ritual · KAM's Friday Y/N file	on you

The whole operation in one line

Monday: data in, plan out. Friday: KAM's Y/N in. Every 4 weeks: models re-earn their job. Every quarter: calendars and budgets refreshed. Every claim: gated, agreed by two engines, glided in 3-point steps, watched by a kill-switch, and graded by the register.